11 4 274

Mechanical-work tools (§ . .)

Revision: 2 · **Created:** 2026-09-08 · **Status:** in use

Rev 2 (2026-09-08) — added `census_query.sh`, the general control-needled census. The library already carried needle discipline in two *specific* tools (`residue_scan.sh`, `anchor_census.sh`), but neither is a general census and **neither takes a negative needle**, so the too-broad half of §11.4.273 — a pattern matching things it should not — had no mechanical guard anywhere. See that tool's section below, and the task-oriented guide at `constitution/docs/scripts/mechanical_tools.md` (§11.4.18).

Six tools that execute the deterministic, decision-free loops an agent would otherwise perform one tool call at a time. They are inherited **by reference** (§11.4.177 / §11.4.28): invoke them from wherever the constitution submodule is checked out; never copy them into a project, and never hardcode a project path into them. Every tool takes its target from an argument.

Why these six exist

Each was extracted from a sequence observed being executed **by hand, more than twice, in a single session** (§11.4.274(a)):

Tool	The loop it replaces	Times done by hand
`mutation_harness.sh`	copy tree → mutate → run suite → read summary → restore → verify	~35
`census_query.sh`	count/grep → hand-check a positive needle → hand-check a negative → decide	every census; 15 instrument errors when skipped
`suite_fanout.sh`	run N suites → tabulate → compare to last round	repeatedly, per review
`residue_scan.sh`	grep for mutation markers, minus the self-reference noise	every pre-commit
`anchor_census.sh`	count anchors per carrier, compare the sets	3 (2 of them with errors)
`await_condition.sh`	poll until a process/log condition holds, bounded	3 ad-hoc rewrites

The boundary (§ . . (d))

These tools **gather and execute**. They do not decide. None of them determines a root cause, renders a review verdict, judges whether a fix is correct, or reads whether evidence supports a claim. `mutation_harness.sh` reports that a suite failed under a named edit — whether that means the gate is genuine is a judgment for the reader. A script that rendered that verdict would be a bluff gate (§11.4.201), which is the failure mode this constitution exists to prevent.

Exit-code vocabulary — shared by every tool (§ . .)

Code	Meaning
`0`	the operation **ran** and the outcome matched expectation
`1`	the operation **ran** and **found something** (residue, drift, a violated expectation)
`2`	the operation **could not run** — bad args, missing target, missing dependency, unparseable output, an instrument that cannot see
`3`	a bounded wait **expired** with the condition still false (`await_condition.sh` only)

`0` , `1` and `2` are deliberately three different facts. "Found nothing", "found something" and "could not look" are not interchangeable, and a tool that collapses them turns a broken instrument into a clean bill of health.

Control needles — how these tools avoid lying (§ . .)

Every tool that **measures** can prove it is able to measure:

- `residue_scan.sh --control-needle STR` — a string that must be findable in the scanned tree. Not found ⇒ exit 2, because an empty result from a scan that cannot see anything is not a clean result. The scan also refuses to report `residue=0` when *every* file was excluded (`examined=0`).
- `anchor_census.sh --control-needle ID` — an anchor that must open a block in every carrier. Zero anchors in any carrier is exit 2, never "all sets identical".
- `mutation_harness.sh` — a mutation whose `--from` text is absent exits 2 rather than reporting the suite result, because a result from an unapplied mutation describes nothing.

- `census_query.sh --positive N --negative N` — **both mandatory**, both repeatable. A missing positive needle, or a *found* negative needle, exits 2 and the count is never printed. This is the only tool here carrying the negative half; the other two prove they can see, not that they are not over-matching.

Two real instrument defects were found this way while building the library, both recorded in `lib/mech_common.sh`: GNU sed 4.9 did not interpret `\x1b`, and under a UTF-8 locale the `[@-~]` bracket range did not match what it appears to. Either one silently disabled ANSI stripping, so every downstream parse ran against coloured text.

`mutation_harness.sh`

Apply one named mutation to a scratch copy of a tree, run one suite, report the counts, restore, and prove the original tree is byte-identical.

```
mutation_harness.sh --tree <dir> --suite <rel> [--name NAME] \
                    [--file <rel> --from <literal> --to <literal>] \
                    [--expect pass|fail] [--env-knob NAME] \
                    [--unset VAR]... [--timeout SECS] [--scratch DIR] [--keep]
```

Omit `--file/--from/--to` for a **baseline** run. Output is one verdict line:

```
MUTATION <name>: EXPECTED-FAIL got <n> failed / <m> passed [OK|VIOLATION]
```

Two traps it is built around

The summary field order flips. Harnesses print `58 passed, 0 failed` on success and `23 failed, 35 passed` on failure — the fields swap. Positional parsing reads a failing run as 23 passes. Each count is located by its own label.

The wrong env knob returns a meaningless green. Suites differ in which variable redirects them at another tree. Setting a name the suite does not read is not an error: the suite falls back to its own location and grades the *real* tree. So by

default the harness runs the suite **from the scratch copy** (no env var involved), and `--env-knob NAME` first asserts `NAME` literally appears in the suite source, exiting 2 if it does not.

`--env-knob` writes into the ORIGINAL tree — know this before using it

The knob's premise is that the suite stays where it lives and is *pointed* elsewhere, so the suite process runs inside `--tree`. A suite that writes an artifact beside itself — a regenerated proof file, a fixture, a cache — therefore writes into the **original** tree, not the copy. Measured 2026-09-08: an env-knob run rewrote a tracked proof file in a repository under independent review. The harness warns before the run, detects it after, refuses to report a result, and **names the changed paths** so they can be restored:

```
SOURCE-TREE MODIFIED: /path/to/tree (before=<md5> after=<md5>)
CHANGED PATHS (restore these):
  ./tests/proof/98-....txt
```

Prefer the default scratch-copy mode. Reach for `--env-knob` only when the suite genuinely cannot be run from a copy, and expect to restore afterwards.

Failure paths it refuses to paper over

`--from` absent · suite output unparseable · suite timed out · source tree changed during the run · restore not byte-identical. All exit 2. None is reported as a suite result.

`census_query.sh`

Run a counting query that proves, in the same command and through the same path, that it can tell PRESENT from ABSENT (§11.4.273).

```
census_query.sh --tree DIR --pattern ERE \
        --positive NEEDLE [--positive NEEDLE]... \
        --negative NEEDLE [--negative NEEDLE]... \
        [--include GLOB]... [--label NAME] [--expect-count N] [--list-files]
```

```
CENSUS harness_in_docs: matches=0 files=0 positive=1/1 negative=0/1 [OK]
```

The two control counts are published on the verdict line, so a reader can see the discipline ran rather than take it on trust. That zero is trustworthy *because* `positive=1/1` says a known-present value was found through this exact query path; without it, an honest zero and a broken grep are the same output.

Needles are ERE, not literal — deliberately. They run through the same matcher as the query, because a control taking a different code path would not prove the query's path can see. Escape metacharacters yourself.

Three refusals, and why each is its own message

Situation	Exit	Message
positive needle absent from the tree	2	`POSITIVE CONTROL FAILED … the instrument cannot see`
positive needle present but outside `--include`	2	`POSITIVE CONTROL OUT OF SCOPE … the CONTROL is misplaced`
negative needle found	2	`NEGATIVE CONTROL FAILED … matches things it must not`

The middle row exists because the two look identical from a bare exit code and have opposite fixes — move the control, or fix the query. Measured during the Phase 0 research that produced this tool: an out-of-scope control reported a working instrument as broken.

Controls are repeatable and **every** member must hold (§11.4.273(f)): searching a two-literal denylist with one literal passes both controls while the answer is wrong, and did — reporting a live credential eliminated while it was still present in four copy-pasteable lines per file.

`--expect-count N` mismatch exits `1` (a finding) and still prints the measured count, which is a different fact from the instrument failing to measure (`2`).

`suite_fanout.sh`

```
suite_fanout.sh --tree <dir> [--baseline FILE] [--out FILE] \
        [--timeout SECS] [--unset VAR]... SUITE_REL...
suite_fanout.sh --tree <dir> --suites-from LIST_FILE ...
```

Runs each suite, tabulates `rc / passed / failed` , writes a TSV usable as the next baseline, and with `--baseline` prints only what moved: `CHANGED` , `MISSING` (in baseline, not run) and `NEW` (run, absent from baseline). A suite that timed out or produced no parseable summary makes the whole run exit 2 — it is never tabulated as a pass.

`residue_scan.sh`

```
residue_scan.sh --tree <dir> [--marker STR]... [--markers-from FILE] \
        [--allow GLOB]... [--allow-from FILE] [--strict-allow] \
        [--control-needle STR] [--no-default-markers]
```

Scans for §11.4.84 mutation residue. Default markers: `MUTATED for paired` , `// always pass` , `# always pass` , `MUTATION-MARKER` , `_mutated_` , `XXX-MUTATION` .

Self-reference is handled as data, not as a hardcoded exception. A file may legitimately contain a marker because it *defines* the marker (a rule, a manual, this scanner) or *plants* one on purpose (a mutation test). Which files those are is a project fact, so they go in an allowlist — `--allow` , `--allow-from` , or a `.mech-residue-allow` file in the scanned tree. Only this tool's own directory is auto-excluded, and the count is published as `self_excluded=N` .

The allowlist is audited in turn. Patterns matching nothing are reported as `STALE-ALLOW`, and `--strict-allow` makes them a finding — so an allowlist cannot quietly grow until it disables the scan.

`anchor_census.sh`

```
anchor_census.sh [--opener-re ERE] [--id-re ERE] [--control-needle ID] \
        [--quiet] CARRIER...
```

Counts anchors that **open a block**, not anchors that are merely mentioned — a carrier cites far more than it declares. Both live opener shapes are matched by default:

```
^### §11.4.N …      canonical constitution heading
^**§11.4.N …        consumer-carrier bolded opener
```

Verifies no id opens two blocks in one carrier (§11.4.227(B)) and that every carrier holds the same set (§11.4.157), printing `only-in <file>: <id>` for each difference. Zero anchors anywhere is exit 2.

The id is taken from the **opener match**, not from the whole line, because a heading routinely cites other anchors after its own. (Measured 2026-09-08 before that was fixed: the canonical constitution reported 900 "anchors" and a wall of phantom duplicates, from 271 real openers.)

Choosing `--opener-re` per carrier class — read this before believing a result

The two default shapes are matched together, which is right when comparing a set of consumer carriers but **over-broad on the canonical constitution**, where a bolded paragraph opener (`**§11.4.30 carve-out.**` , `**§11.4.93 amendment.**`) is a cross-reference and not a declaration. Narrow it:

```
# canonical constitution — declarations are '### §' only
anchor_census.sh --opener-re '^### §11\.4\.[0-9]+' --control-needle 11.4.274 Constitution.md

# a sub-anchor such as §11.4.10.A needs a widened id regex, or it collapses onto
# its parent and is reported as a duplicate
anchor_census.sh --opener-re '^### §11\.4\.[0-9]+(\.[A-Z])?' --id-re '11\.4\.[0-9]+(\.[A-Z])?'
                 Constitution.md
```

Measured 2026-09-08 with those flags: `Constitution.md → 242 anchors, no duplicates, exit 0`. Both behaviours are covered by tests rather than only described here.

`await_condition.sh`

```
await_condition.sh --until 'SHELL CMD' [--timeout SECS] [--interval SECS] [--label NAME]
await_condition.sh --pid-gone PID ...
await_condition.sh --log-contains FILE --pattern ERE ...
```

Exactly one outcome line, and a timeout that **cannot** read as success:

```
AWAIT <label>: SATISFIED after <n>s (polls=<k>)
AWAIT <label>: TIMEOUT after <n>s (polls=<k>) — condition never held
```

`--pid-gone` uses `kill -0`, which tests existence without signalling; this tool never signals a process (§11.4.174). The condition is re-evaluated once after the ceiling, so a condition that became true during the final sleep is not misreported as a timeout.

Tests

```
constitution/scripts/mechanical/tests/run_all.sh
```

212 assertions across seven suites, hermetic (each builds its own fixture tree and its own suites), with no dependency on any consuming project's harness. Every tool's failure paths, exit-code semantics and control needles are covered. Measured 2026-09-08: `ALL SUITES: 212 passed, 0 failed` (181 before `census_query.sh` landed), identical across consecutive runs (§11.4.50).

A consuming project verifies these tools from its own side. These suites are deliberately project-agnostic, so an acceptance test reproducing a *measured outcome of a specific repository* belongs to that repository — reaching out to the constitution, never the reverse. Reference example: `claude_toolkit/scripts/tests/test_mechanical_tools_acceptance.sh`, which asserts each tool's refusal paths, degrades honestly to a named SKIP when no constitution checkout is reachable, and — opt-in via `CMA_MECH_ACCEPTANCE=1` — reproduces that repo's own measured baseline and a named mutation against it.

Re-scan cadence (§ . . (c))

Extraction is a standing obligation, not a cleanup. New mechanical loops accrete as features land. Periodically re-read recent transcripts and scripts for recurring command sequences and "run these in order" recipes, and extract what you find. This directory is where the results go.